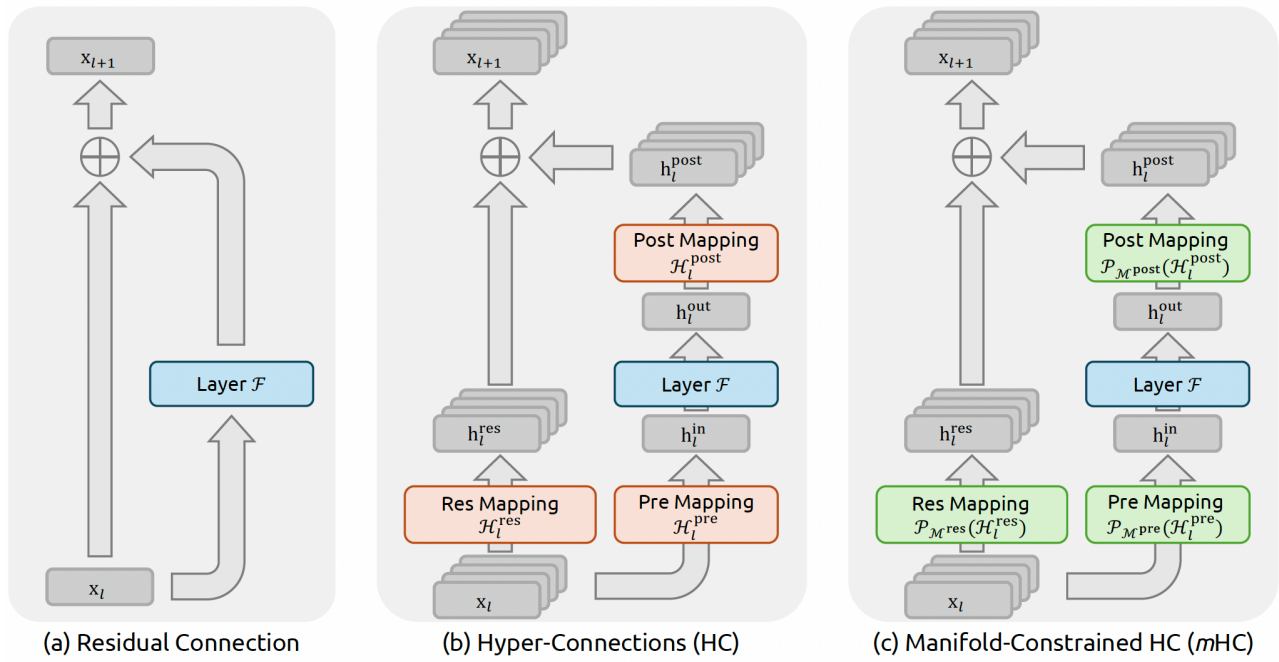


mHC

 [DeepSeek_mHC– Manifold–Constrained Hyper–Connections.pdf \(635 kB\)](#) (2026 DS: Zhenda Xie)

Architecture



Algo

- ResNet: $x_{l+1} = x_l + F(x_l, w_l)$, where x_l is R^C
- HC: introduced expansion rate: n ,

$x_{l+1} = H_l^{\text{res}} * x_l + H_l^{\text{post}T} * F(H_l^{\text{pre}} * x_l, w_l)$, where x_l is R^{n*C} , H_l^{pre} and H_l^{post} is R^{1*n} , H_l^{res} is R^{n*n} $\rightarrow$ final output is R^{n*c}

Formally, HC computes the coefficients as follows:

$$\begin{cases} \tilde{x}_l = \text{RMSNorm}(x_l) \\ \mathcal{H}_l^{\text{pre}} = \alpha_l^{\text{pre}} \cdot \tanh(\theta_l^{\text{pre}} \tilde{x}_l^T) + \mathbf{b}_l^{\text{pre}} \\ \mathcal{H}_l^{\text{post}} = \alpha_l^{\text{post}} \cdot \tanh(\theta_l^{\text{post}} \tilde{x}_l^T) + \mathbf{b}_l^{\text{post}} \\ \mathcal{H}_l^{\text{res}} = \alpha_l^{\text{res}} \cdot \tanh(\theta_l^{\text{res}} \tilde{x}_l^T) + \mathbf{b}_l^{\text{res}}, \end{cases} \quad (5)$$

- mHC: motivation is to solve the instability of HC during training and mitigate the I/O bottleneck and GPU memory increasing $\rightarrow$ introduced manifold-constrained

- manifold-constrained mainly is the "final constrained mapping part" below → doubly stochastic matrices (the word "manifold" represents a geometric space in math)

In this section, we detail the calculation process of $\mathcal{H}_l^{\text{pre}}$, $\mathcal{H}_l^{\text{post}}$, and $\mathcal{H}_l^{\text{res}}$ in $m\text{HC}$. Given the input hidden matrix $\mathbf{x}_l \in \mathbb{R}^{n \times C}$ at the l -th layer, we first flatten it into a vector $\tilde{\mathbf{x}}_l = \text{vec}(\mathbf{x}_l) \in \mathbb{R}^{1 \times nC}$ to preserve full context information. Then, we follow the original HC formulation to get the dynamic mappings and the static mappings as follows:

$$\begin{cases} \tilde{\mathbf{x}}'_l = \text{RMSNorm}(\tilde{\mathbf{x}}_l) \\ \tilde{\mathcal{H}}_l^{\text{pre}} = \alpha_l^{\text{pre}} \cdot (\tilde{\mathbf{x}}'_l \varphi_l^{\text{pre}}) + \mathbf{b}_l^{\text{pre}} \\ \tilde{\mathcal{H}}_l^{\text{post}} = \alpha_l^{\text{post}} \cdot (\tilde{\mathbf{x}}'_l \varphi_l^{\text{post}}) + \mathbf{b}_l^{\text{post}} \\ \tilde{\mathcal{H}}_l^{\text{res}} = \alpha_l^{\text{res}} \cdot \text{mat}(\tilde{\mathbf{x}}'_l \varphi_l^{\text{res}}) + \mathbf{b}_l^{\text{res}}, \end{cases} \quad (7)$$

where $\varphi_l^{\text{pre}}, \varphi_l^{\text{post}} \in \mathbb{R}^{nC \times n}$ and $\varphi_l^{\text{res}} \in \mathbb{R}^{nC \times n^2}$ are linear projections for dynamic mappings and $\text{mat}(\cdot)$ is a reshape function from $\mathbb{R}^{1 \times n^2}$ to $\mathbb{R}^{n \times n}$.

Then, the final constrained mappings are obtained via:

$$\begin{cases} \mathcal{H}_l^{\text{pre}} = \sigma(\tilde{\mathcal{H}}_l^{\text{pre}}) \\ \mathcal{H}_l^{\text{post}} = 2\sigma(\tilde{\mathcal{H}}_l^{\text{post}}) \\ \mathcal{H}_l^{\text{res}} = \text{Sinkhorn-Knopp}(\tilde{\mathcal{H}}_l^{\text{res}}), \end{cases} \quad (8)$$

where $\sigma(\cdot)$ denotes the Sigmoid function. The Sinkhorn-Knopp($\cdot$) operator firstly makes all elements to be positive via an exponent operator and then conducts iterative normalization process that alternately rescales rows and columns to sum to 1. Specifically, given a positive matrix $\mathbf{M}^{(0)} = \exp(\tilde{\mathcal{H}}_l^{\text{res}})$ as the start point, the normalization iteration proceeds as:

$$\mathbf{M}^{(t)} = \mathcal{T}_r \left(\mathcal{T}_c(\mathbf{M}^{(t-1)}) \right), \quad (9)$$

where $\mathcal{T}_r$ and $\mathcal{T}_c$ denote row and column normalization, respectively. This process converges to a doubly stochastic matrix $\mathcal{H}_l^{\text{res}} = \mathbf{M}^{(t_{\max})}$ as $t_{\max} \rightarrow \infty$. We choose $t_{\max} = 20$ as a practical value in our experiments.

Infra Design

- Due to that calculation of RMSNorm brings in significant latency → reordered the normalization process and matrix manipulation

- Specifically,

$$\varphi_l : \text{tfloat32} \quad [nC, n^2 + 2n] \quad (10)$$

$$\tilde{\mathbf{x}}_l : \text{bfloat16} \quad [1, nC] \quad (11)$$

$$\alpha_l^{\text{pre}}, \alpha_l^{\text{post}}, \alpha_l^{\text{res}} : \text{float32} \quad \text{Scalars} \quad (12)$$

$$\mathbf{b}_l : \text{float32} \quad [1, n^2 + 2n] \quad (13)$$

$$\left[\tilde{\mathcal{H}}_l^{\text{pre}}, \tilde{\mathcal{H}}_l^{\text{post}}, \tilde{\mathcal{H}}_l^{\text{res}} \right] : \text{float32} \quad = \tilde{\mathbf{x}}_l \varphi_l \quad (14)$$

$$r : \text{float32} \quad = \|\tilde{\mathbf{x}}_l\|_2 / \sqrt{nC} \quad (15)$$

$$\left[\tilde{\mathcal{H}}_l^{\text{pre}}, \tilde{\mathcal{H}}_l^{\text{post}}, \tilde{\mathcal{H}}_l^{\text{res}} \right] : \text{float32} \quad = 1/r \left[\alpha_l^{\text{pre}} \tilde{\mathcal{H}}_l^{\text{pre}}, \alpha_l^{\text{post}} \tilde{\mathcal{H}}_l^{\text{post}}, \alpha_l^{\text{res}} \tilde{\mathcal{H}}_l^{\text{res}} \right] + \mathbf{b}_l \quad (16)$$

$$\mathcal{H}_l^{\text{pre}} : \text{float32} \quad = \sigma \left(\tilde{\mathcal{H}}_l^{\text{pre}} \right) \quad (17)$$

$$\mathcal{H}_l^{\text{post}} : \text{float32} \quad = 2\sigma \left(\tilde{\mathcal{H}}_l^{\text{post}} \right) \quad (18)$$

$$\mathcal{H}_l^{\text{res}} : \text{float32} \quad = \text{Sinkhorn-Knopp} (\tilde{\mathcal{H}}_l^{\text{res}}) \quad (19)$$

- Applied 2 extra kernels to reduce read/write elements: $F_{pre} := H_l^{\text{pre}} * x_l$;
 $F_{post,res} := H_l^{\text{res}} * x_l + H_l^{\text{post}T} * F$
- The n-stream residual in mHC brings in substantial communication latency across pipeline stages compared to single-stream, to mitigate this problem, the author adopted and enhanced **DualPipe schedule** → to prevent blocking of communication stream, **high prioritizing** $F_{post,res}$: kernels of MLPcomputation → improved the overlapping of communication and computation in pipeline (不是很懂..)

- Specifically,

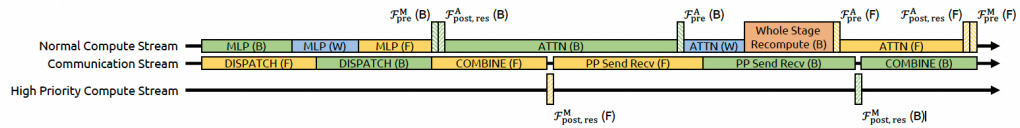


Figure 4 | **Communication-Computation Overlapping for mHC.** We extend the DualPipe schedule to handle the overhead introduced by mHC. Lengths of each block are illustrative only and do not represent actual duration. (F), (B), (W) refers to forward pass, backward pass, weight gradient computation, respectively. $\mathcal{F}^A$ and $\mathcal{F}^M$ represents kernels corresponded to Attention and MLP, respectively.